



UNIVERSIDAD DE
COSTA RICA
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Examen Parcial I: Plataforma de Triage y Citas Médicas "MedTriage Express" (Modalidad Sin DBMS Local / H2 Database)

Ficha Técnica del Examen

- **Fecha:** Lunes 12 de Octubre, 2026
- **Duración Máxima:** 3 horas (180 minutos)
- **Ponderación:** 20% de la nota final del curso
- **Modalidad:** Presencial (Laboratorio de Cómputo 3) - Individual
- **Motor de Base de Datos:** H2 Database en Memoria (Spring Data JPA)
- **Entornos a Desarrollar:**

- Backend: Spring Boot 3 (versión de Java disponible en su computadora) en carpeta `medtrriage-backend`
- Frontend: Angular 19 Standalone en carpeta `medtrriage-frontend`
- **Restricciones:** Prohibición estricta de asistentes de IA generativa (ChatGPT, Copilot, Claude, Gemini). Prohibido el acceso a documentación externa o repositorios previos. Conexión obligatoria a GitHub con commits semánticos periódicos.

Introducción y Caso de Negocio: MedTriage Express

El centro de salud **MedTriage Express** requiere un sistema informático de pila completa (Full-Stack) para gestionar el flujo de recepción, prioridad de triaje y asignación de citas médicas de emergencia.

Usted ha sido contratado para construir desde cero tanto la **API RESTful back-end en Spring Boot** como la **interfaz cliente SPA en Angular 19 Standalone**. En esta variante de examen (sin servidor local de SQL Server disponible), usted configurará una **base de datos H2 en memoria** en Spring Boot utilizando scripts de inicialización ANSI SQL (`schema.sql` y `data.sql`).

Configuración e Inicialización de Base de Datos H2 (En Memoria)

Al no disponer de un motor SQL Server local en el laboratorio, usted debe configurar la persistencia en memoria dentro del proyecto `medtrriage-backend`.

1. Dependencia en `pom.xml`

Asegúrese de incluir la dependencia de H2 Database en el archivo `pom.xml`:

```
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
```

```
</dependency>
```

2. Configuración en `src/main/resources/application.properties`

Agregue las siguientes propiedades para activar H2, la consola web y la ejecución de scripts al iniciar la aplicación (desactivando la generación automática de Hibernate para evitar duplicidad de tablas):

```
spring.datasource.url=jdbc:h2:mem:medtriagedb;DB_CLOSE_DELAY=-1
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect

# Desactivar auto-ddl de Hibernate para evitar conflictos con schema.sql
spring.jpa.hibernate.ddl-auto=none

# H2 Console y activacion de inicializacion SQL
spring.h2.console.enabled=true
spring.sql.init.mode=always
```

3. Script de Esquema (`src/main/resources/schema.sql`)

Cree el archivo `schema.sql` en la carpeta `src/main/resources/` con el siguiente contenido DDL:

```
DROP TABLE IF EXISTS cita_medica;
DROP TABLE IF EXISTS paciente;
DROP TABLE IF EXISTS doctor;

CREATE TABLE doctor (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    especialidad VARCHAR(80) NOT NULL,
    disponible BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE paciente (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    identificacion VARCHAR(20) NOT NULL UNIQUE,
    nombre_completo VARCHAR(120) NOT NULL,
    correo VARCHAR(100) NOT NULL,
```

```

    telefono VARCHAR(20) NOT NULL
);

CREATE TABLE cita_medica (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    codigo_cita VARCHAR(30) NOT NULL UNIQUE,
    paciente_id BIGINT NOT NULL,
    doctor_id BIGINT NOT NULL,
    nivel_prioridad VARCHAR(20) NOT NULL,
    motivo_consulta VARCHAR(255) NOT NULL,
    estado VARCHAR(20) NOT NULL,
    monto_consulta DECIMAL(10,2) NOT NULL,
    fecha_cita TIMESTAMP NOT NULL,
    CONSTRAINT fk_cita_paciente
        FOREIGN KEY (paciente_id) REFERENCES paciente(id),
    CONSTRAINT fk_cita_doctor
        FOREIGN KEY (doctor_id) REFERENCES doctor(id)
);

```

4. Datos Semilla (src/main/resources/data.sql)

Cree el archivo data.sql en la carpeta src/main/resources/ para cargar los registros iniciales automáticamente al arrancar la aplicación:

```

INSERT INTO doctor (nombre, especialidad, disponible) VALUES
('Dr. Carlos Alvarado', 'Medicina General', TRUE),
('Dra. Sofia Segura', 'Pediatría', TRUE),
('Dr. Roberto Mora', 'Urgencias', FALSE);

INSERT INTO paciente (identificacion, nombre_completo, correo, telefono)
VALUES
('118230495', 'Elena Madrigal Castro', 'elena@ucr.ac.cr', '88776655'),
('207410982', 'Mario Jimenez Solano', 'mario@ucr.ac.cr', '83332211');

INSERT INTO cita_medica
(codigo_cita, paciente_id, doctor_id, nivel_prioridad,
motivo_consulta, estado, monto_consulta, fecha_cita)
VALUES
('CIT-2026-001', 1, 1, 'ALTA',
'Fiebre alta y dificultad respiratoria', 'PENDIENTE',
25000.00, CURRENT_TIMESTAMP),
('CIT-2026-002', 2, 2, 'MEDIA',
'Dolor abdominal moderado', 'EN_ATENCION',
20000.00, CURRENT_TIMESTAMP);

```

Requerimientos del Examen

1. Desarrollo Back-End (medtriage-backend - 35%)

Cree un proyecto Spring Boot 3 en la versión de Java disponible en su computadora estructurado en las siguientes capas:

- **[] Mapeo JPA (com.medtriage.model):**

Implemente las entidades `Doctor.java`, `Paciente.java` y `CitaMedica.java` con las relaciones `@ManyToOne` hacia `Paciente` y `Doctor`.

[] DTOs (com.medtriage.dto):

- `CitaMedicaDTO`: Campos `id`, `codigoCita`, `nombrePaciente`, `nombreDoctor`, `nivelPrioridad`, `motivoConsulta`, `estado`, `montoConsulta`, `fechaCita`.

`CrearCitaDTO`: Campos `pacienteId`, `doctorId`, `nivelPrioridad` (ALTA, MEDIA, BAJA), `motivoConsulta`, `montoConsulta`.

[] Persistencia (com.medtriage.repository):

`CitaMedicaRepository`: Métodos JPA para buscar por `nivelPrioridad` y ordenar descendientemente por `fechaCita`.

[] Lógica de Negocio (com.medtriage.service):

`CitaMedicaService`:

- `obtenerTodas()`: Retorna lista de DTOs.
- `obtenerPorPrioridad(String prioridad)`: Retorna citas por prioridad.
- `crearCita(CrearCitaDTO dto)`: Genera código único `CIT-2026-XXXX`, valida disponibilidad del doctor, establece estado en `PENDIENTE` y guarda en la base de datos.
- `actualizarEstado(Long id, String nuevoEstado)`: Cambia el estado de la cita (`EN_ATENCION`, `ATENDIDA`, `CANCELADA`).

[] Controlador RESTful (com.medtriage.controller):

- Controller `@RestController` en `/api/v1/citas`.
 - Habilite CORS con `@CrossOrigin(origins = "http://localhost:4200")`.
 - Endpoints: `GET /api/v1/citas`, `GET /api/v1/citas/prioridad/{prioridad}`, `POST /api/v1/citas`, `PATCH /api/v1/citas/{id}/estado`.
-

2. Pruebas Unitarias y Manejo de Excepciones (15%)

- **[] Manejador de Excepciones Centralizado:**

Implemente `@RestControllerAdvice` en `com.medtriage.exception` retornando la norma RFC 7807 (Problem Details).

[] Prueba Unitaria con Mockito:

- En `src/test/java/com/medtriage/service/CitaMedicaServiceTest.java`, construya al menos una prueba unitaria probando el método `crearCita()` aislando el repositorio con `@Mock` y `@InjectMocks`.
-

3. Desarrollo Front-End Angular 19 (`medtriage-frontend` - 35%)

Inicialice el cliente con `ng new medtriage-frontend --standalone (CSS format, SSR No)`:

- **[] Configuración Global (`app.config.ts` y `environment.ts`):**
- Registre `provideHttpClient(withFetch())` en `app.config.ts`.

Defina `API_URL: 'http://localhost:8080/api/v1/'` en `environment.ts`.

[] Modelos e Interfaz (`src/app/models/cita.model.ts`):

Interfaces `CitaMedica`, `Doctor`, `Paciente` y `CrearCitaPayload`.

[] Servicio Angular (`src/app/services/cita.service.ts`):

- Inyección mediante `inject(HttpClient)`.

Métodos `getCitas()`, `getCitasPorPrioridad(prioridad)`, `crearCita(payload)`, `actualizarEstado(id, estado)`.

[] Componente Dashboard con Angular Signals (`CitaListComponent`):

- **Ruta:** `/citas`
- Uso de Signals (`signal()`, `computed()`) para almacenar la lista de citas y aplicar un filtro reactivo por prioridad (TODAS, ALTA, MEDIA, BAJA).

Tabla responsiva con badges de color por estado y botón interactivo para avanzar el estado de la cita.

[] Componente Formulario de Triage (`CitaFormComponent`):

- **Ruta:** `/nueva-cita`
- Formulario con `[(ngModel)]` para registrar paciente, doctor, prioridad, motivo y monto.

Procesamiento y envío mediante `CitaService`.

[] Navegación y Rutas (`app.routes.ts`):

- Enrutamiento entre `/citas` y `/nueva-cita` con barra de navegación superior en `app.component.html`.

4. Flujo de Trabajo en Git y Commits Semánticos (15%)

- [] Repositorio inicializado con `.gitignore` adecuado.
- [] Mínimo 5 commits semánticos individuales durante el desarrollo de la prueba (ejemplos: `feat(backend): add JPA entities`, `feat(angular): implement CitaListComponent with Signals`).

Distribución Recomendada del Tiempo (180 Minutos)

Bloque de Tiempo	Fase de Desarrollo	Duración
Bloque 1	Configurar H2 Database (properties, schema.sql, data.sql) y crear proyectos base	20 min
Bloque 2	Desarrollo de Backend (Entidades, DTOs, Service, Controller)	50 min
Bloque 3	Manejo de Excepciones RFC 7807 y Prueba Unitaria JUnit/Mockito	20 min
Bloque 4	Setup de Angular 19, environment.ts, Models y CitaService	20 min
Bloque 5	Componentes Standalone (CitaListComponent con Signals y CitaFormComponent)	50 min
Bloque 6	Integración Full-Stack, verificación final y Commits en GitHub	20 min

Rúbrica de Evaluación (100 Puntos)

Criterio	Puntos	Descripción
----------	--------	-------------

Arquitectura Back-End RESTful	35 pts	Mapeo JPA correcto en H2 Database, capas DTO/Service/Controller, validaciones de negocio y política CORS habilitada.
Front-End Angular 19 SPA	35 pts	Componentes Standalone enrutados, reactividad con Angular Signals (<code>signal()</code> , <code>computed()</code>), consumo HTTP y Data Binding.
Excepciones & Pruebas Unitarias	15 pts	Formateo RFC 7807 con <code>@RestControllerAdvice</code> y prueba unitaria en Mockito ejecutando exitosamente.
Git & Versionamiento	15 pts	Commits semánticos periódicos durante el examen en la cuenta personal de GitHub del estudiante.